

Příprava na test

Objektové programování 2 – UAI/520

Zpracováno z prezentací a vzorového testu Verity

Obsah

1	Práce s chybami a výjimky	3
1.1	Základní terminologie	3
1.2	Kategorie výjimek	3
1.3	Vyhazování výjimek	3
1.4	Zpracování výjimek --try-catch-finally	4
1.5	Definice vlastních výjimek	4
1.6	Aserce (Assertions)	4
1.7	Zotavení z nestandardních stavů	4
2	Genericita (Generics)	5
2.1	Základní koncepty	5
2.2	Přínosy genericity	5
2.3	Meze typových parametrů	6
2.4	Generické metody	6
2.5	Vymazání typu (Type Erasure)	6
2.6	Wildcard capture	7
3	Tvorba grafického rozhraní (Swing)	7
3.1	Architektura GUI	7
3.2	Struktura aplikačního okna	7
3.3	Manažeři rozvrhování (Layout Managers)	8
3.4	Zpracování událostí	8
3.5	Vnořené třídy (Nested Classes)	8
3.6	Dialogy	9
4	JavaFX	9
4.1	Historie a dostupnost	9
4.2	Architektura JavaFX	9
4.3	Klíčové rysy JavaFX	9
4.4	Graf scény (Scene Graph)	10
4.5	FXML a Controller	10
4.6	Layouty v JavaFX	11
4.7	Životní cyklus JavaFX aplikace	11
4.8	Pracovní vlákna v JavaFX	11
5	Vlákna v Javě (Concurrency)	12
5.1	Procesy vs. vlákna	12
5.2	Vytvoření a spuštění vlákna	12
5.3	Stavy vlákna	12
5.4	Přerušování vlákna (Interrupt)	13
5.5	Synchronizace	13
5.5.1	Problémy souběhu	13
5.5.2	Nástroje synchronizace	13
5.5.3	Guarded Blocks (wait/notify)	14
5.6	Problémy souběhu – Deadlock, Starvation, Livelock	14
5.7	java.util.concurrent	14

5.7.1	Executor framework	15
5.7.2	Fork/Join framework	15
5.7.3	Konkurentní kolekce	15
5.8	Concurrency ve Swing	16
5.8.1	SwingWorker	16
6	Shrnutí klíčových otázek ze vzorového testu	16
6.1	Odpovědi na viditelné otázky	16
7	Rychlá reference – často kladené otázky	17

1 Práce s chybami a výjimky

1.1 Základní terminologie

Defenzivní programování

Předvídání, že se věci mohou pokazit. Server by měl předpokládat, že klienti jsou **potenciálně nepřátelští**, ne že se chovají dobře.

Typické chybové situace:

- Nesprávná implementace (nevyhovuje specifikaci)
- Nepatřičná žádost o objekt (neplatný index)
- Nekonzistentní nebo nevhodný stav objektu
- Problémy dané prostředím: nesprávné URL, přerušení sítě, chybějící soubory, nedostatečná oprávnění

1.2 Kategorie výjimek

Typ	Použití	Příklad
Kontrolované (checked)	Podtřída <code>Exception</code> . Předvídatelná selhání, zotavení možné.	<code>IOException</code> , <code>FileNotFoundException</code>
Nekontrolované (unchecked)	Podtřída <code>RuntimeException</code> . Nepředvídatelná selhání, zotavení nepravděpodobné.	<code>IllegalArgumentException</code> , <code>NullPointerException</code>

Klíčové rozdíly

- **Kontrolované:** Kompilátor **vynucuje** zpracování (try-catch nebo throws). Používají se pro **externí** problémy (soubory, síť).
- **Nekontrolované:** Kompilátor **nekontroluje**. Používají se pro **programátorské** chyby. Způsobí ukončení programu, pokud nejsou odchyceny.

1.3 Vyhazování výjimek

```
public ContactDetails getDetails(String key) {
    if(key == null) {
        throw new IllegalArgumentException("null key in getDetails");
    }
    if(key.trim().length() == 0) {
        throw new IllegalArgumentException("Empty key passed to getDetails");
    }
    return book.get(key);
}
```

Důležité:

- Výjimka přerušuje normální tok řízení – metoda se předčasně ukončí, nevrací hodnotu
- Klient může výjimku odchytit (catch) nebo nechat propagovat
- Dokumentace: používejte `@throws` v Javadoc

1.4 Zpracování výjimek – try-catch-finally

```
try {
    addressbook.saveToFile(filename);
    successful = true;
} catch(IOException e) {
    System.out.println("Unable to save to " + filename);
    successful = false;
} finally {
    // Proveďte se VŽDY -- i při return v try/catch
    // Uzavření souborů, uvolnění zdrojů
}
```

Vícenásobné odchycení:

```
try { ... }
catch(EOFException | FileNotFoundException e) { ... } // Java 7+
```

1.5 Definice vlastních výjimek

```
public class NoMatchingDetailsException extends Exception {
    private String key;
    public NoMatchingDetailsException(String key) { this.key = key; }
    public String getKey() { return key; }
    public String toString() {
        return "No details matching '" + key + "' were found.";
    }
}
```

Pravidlo pro výběr nadtypu

- Rozšiř RuntimeException → nekontrolovaná
- Rozšiř Exception → kontrolovaná

1.6 Aserce (Assertions)

Assertions vs. Výjimky

NEPLATÍ: Aserce nejsou alternativou k výjimkám!

- Aserce: **interní kontroly** konzistence, používají se ve vývoji, odstraňují se z produkce
- Výjimky: **externí** problémy, zůstávají v produkčním kódu

```
assert !keyInUse(key); // kontrola po removeDetails
assert consistentSize() : "Nekonzistentní velikost adresáře";
```

Pozor: assert se vypíná v produkci (-ea flag). **Nezahrnuj** do assert normální funkcionalitu!

1.7 Zotavení z nestandardních stavů

```
boolean successful = false;
int attempts = 0;
do {
    try {
```

```

        contacts.saveToFile(filename);
        successful = true;
    } catch(IOException e) {
        attempts++;
        if(attempts < MAX_ATTEMPTS) {
            filename = alternativniJmenoSouboru;
        }
    }
} while(!successful && attempts < MAX_ATTEMPTS);

```

Strategie vyhýbání se selháním:

- Použijte dotazovací metody serveru (např. `keyInUse()`) před voláním rizikové operace
- Nekontrolované výjimky pro skutečně neočekávané stavy

2 Genericita (Generics)

2.1 Základní koncepty

Terminologie genericity

Typový parametr	Zástupný znak (např. T, E, K, V)
Typový argument	Konkrétní typ dosazený za parametr
Instance generického typu	Generický typ po dosazení argumentu
Raw typ	Použití bez typového argumentu (<code>List</code> místo <code>List<String></code>)

Jmenné konvence:

Značka	Význam
E	Element (kolekce)
K	Key (klíč)
V	Value (hodnota)
T	Type (obecný typ)
N	Number
S, U, V	Další typové parametry

2.2 Přínosy genericity

1. **Vyšší vypovídací schopnost** – kód je čitelnější
2. **Eliminace přetypování** – žádné `(Integer) list.get(0)`
3. **Silnější typová kontrola při překladu** – chyby se odhalí dříve
4. **Možnost užití generických algoritmů** – odstranění opakujícího se kódu

Nebezpečí raw typů

`Box rawBox = new Box();` – kompilátor neví o typu, varování při volání generických metod. Může vést k chybám při běhu (`ClassCastException`).

2.3 Meze typových parametrů

```
// Horní omezení (extends)
public class NaturalNumber<T extends Integer> { ... }

// Vícenásobná omezení -- třída musí být první!
class D<T extends A & B & C> { ... }

// Wildcards
?           // libovolný typ (není Object!)
? extends Number // podtyp Number (čtení)
? super Integer  // nadtyp Integer (zápis)
```

PECS pravidlo

Producer Extends, Consumer Super:

- Čteš z kolekce? → ? extends T
- Zapisuješ do kolekce? → ? super T

2.4 Generické metody

```
// Typový parametr před návratovým typem
public static <K, V> boolean compare(Pair<K, V> p1, Pair<K, V> p2) {
    return p1.getKey().equals(p2.getKey()) &&
           p1.getValue().equals(p2.getValue());
}

// Omezení na Comparable
public static <T extends Comparable<T>> int countGreaterThan(T[] arr, T elem) {
    int count = 0;
    for (T e : arr)
        if (e.compareTo(elem) > 0)
            ++count;
    return count;
}
```

2.5 Vymazání typu (Type Erasure)

Co je type erasure?

Překladač přepíše všechny typové parametry jejich mezemi nebo `Object` (pokud meze nejsou).
V bajtkódu **nejsou** generické typy!

```
// Zdrojový kód:
public class Node<T> {
    private T data;
    public T getData() { return data; }
}

// Po erasure:
public class Node {
    private Object data; // nebo Comparable s extends
    public Object getData() { return data; }
}
```

Důsledky:

- Nelze vytvářet instance typových parametrů: `new E()` – CHYBA
- Nelze deklarovat statické proměnné typového parametru
- Nelze vytvářet pole parametrických typů: `new List<Integer>[2]` – CHYBA
- Nelze přetížit metody, které mají stejnou signaturu po erasure

2.6 Wildcard capture

```
// PROBLÉM: překladač neumí odvodit typ ?
void foo(List<?> i) {
    i.set(0, i.get(0)); // CHYBA: get vrací Object, set očekává ?
}

// ŘEŠENÍ: pomocná metoda
void foo(List<?> i) {
    fooHelper(i);
}
private <T> void fooHelper(List<T> l) {
    l.set(0, l.get(0)); // OK: T je odvozeno
}
```

3 Tvorba grafického rozhraní (Swing)**3.1 Architektura GUI****Tři pilíře GUI**

1. **Komponenty** – stavební bloky (tlačítka, menu, posuvníky)
2. **Rozvržení** – manažeři rozvrhování (Layout Managers)
3. **Události** – reakce na uživatelský vstup

Hierarchie balíčků:

- `java.awt` – základní komponenty (AWT)
- `javax.swing` – rozšířené komponenty (Swing) – **používej tyto!**

3.2 Struktura aplikačního okna

Komponenta	Účel
<code>JFrame</code>	Hlavní aplikační okno
<code>JMenuBar</code>	Pruh menu pod titulkem
<code>JMenu</code>	Menu (např. File)
<code>JMenuItem</code>	Položka menu (např. Open)
<code>JPanel</code>	Kontejner pro seskupení komponent
<code>JLabel</code>	Textový/popiskový prvek
<code>JButton</code>	Tlačítko
<code>TextField</code>	Vstupní textové pole

3.3 Manažeři rozvrhování (Layout Managers)

Layout	Chování
FlowLayout	Komponenty vedle sebe, zalamování jako text
BorderLayout	5 oblastí: NORTH, SOUTH, EAST, WEST, CENTER
GridLayout	Mřížka s pevným počtem řádků a sloupců
BoxLayout	Vertikální nebo horizontální řada
GridBagLayout	Nejsložitější, pružné umístění

Vnořené kontejnery

Pro sofistikované rozvržení použij **vnořené panely** (JPanel) s různými layouty. Často vhodnější než GridBagLayout!

3.4 Zpracování událostí

Přístupy:

1. **Centralizovaný** – jeden objekt implementuje ActionListener, rozhoduje podle `getActionCommand()`
 - Nevýhoda: špatně škáluje, závislost na textu komponenty
2. **Anonymní vnitřní třídy** – každá komponenta má vlastní listener

```
openItem.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) { openFile(); }
});
```

3. **Lambda výrazy** (Java 8+) – nejčistší zápis

```
openItem.addActionListener(e -> openFile());
```

3.5 Vnořené třídy (Nested Classes)

Typ	Instance	Přístup k vnější třídě
Statická vnořená	Nezávislá na instanci vnější	Pouze statické členy
Vnitřní (inner)	Vázána na instanci vnější	Všechny členy, i private
Lokální vnitřní	V bloku metody	Všechny členy
Anonymní vnitřní	Bez hlavičky, jedna instance	Dědí z nadtypu/implements interface

Anonymní vnitřní třída

Deklarace bez jména, typ určen nadtypem. Používá se pro **jednorázovou** instanci listeneru.

```
// Syntaxe: new Supertyp() { ... implementace ... }
new ActionListener() {
    public void actionPerformed(ActionEvent e) { ... }
}
```


3.6 Dialogy

- **Modální** – blokují ostatní interakce (vyžadují odpověď)
- **Nemodální** – umožňují další interakci (pozor na nekonzistence)
- `JOptionPane` – standardní dialogy: Message, Confirm, Input

4 JavaFX

4.1 Historie a dostupnost

Rok	Verze
1996	AWT (JDK 1.0)
1998	Swing (J2SE 1.2)
2007	JavaFX 1.0 (JavaFX Script)
2011	JavaFX 2.0 (ukončení Script, čistá Java)
2014	JavaFX 8 (součást Java 8)
2018+	Java 9+: samostatné SDK (OpenJFX, Gluon)

4.2 Architektura JavaFX

Hierarchie vrstev (odspoda nahoru)

1. **JVM** – Java Virtual Machine
2. **JDK API** – základní knihovny
3. **Prism** – grafický engine (2D/3D, hardware acceleration)
4. **Glass Windowing Toolkit** – platformově závislá vrstva
5. **Quantum Toolkit** – spojuje Prism a Glass
6. **JavaFX Public APIs + Scene Graph** – vrstva pro vývojáře

4.3 Klíčové rysy JavaFX

- FXML a Scene Builder – deklarativní popis GUI
- CSS stylování – vzhled oddělen od logiky
- WebView – vkládání webových stránek (WebKit)
- Interoperabilita se Swingem (**SwingNode**)
- 3D grafika, Canvas API, Printing API, Rich Text
- Multitouch, Hi-DPI, hardwarová akcelerace (Prism)

4.4 Graf scény (Scene Graph)

Struktura GUI JavaFX

Stage $\supset$ Scene $\supset$ Root Pane $\supset$ Nodes

- **Stage** – “jeviště”, vnější obálka (okno aplikace)
- **Scene** – “scéna”, obsah okna
- **Root Pane/Container** – StackPane, GridPane, BorderPane, FlowPane...
- **Nodes** – tlačítka, texty, obrazce, 3D objekty

Hierarchie elementů (od nejvyšší):

Stage > Scene > Pane/Container > Controls/Nodes

Otázka ze vzorového testu!

Který objekt je **nejvyšše postavený** v hierarchii? → **Stage** (nikoli Scene, Pane ani GridLayout)

4.5 FXML a Controller

Struktura FXML aplikace

FXMLDocument.fxml	Deklarativní popis GUI (generuje Scene Builder)
FXMLDocumentController.java	Logika interakce, anotace @FXML
HelloWorldFXML.java	Spouštěcí třída (Application.launch())

Účel anotace @FXML

@FXML slouží k **propojení identifikace komponent** mezi kontrolerem a definicí vzhledu GUI. Umožňuje injectovat komponenty z FXML do controlleru podle fx:id. **Nejedná se o:** generování dokumentace, označení jazyka FXML.

```
public class FXMLDocumentController implements Initializable {
    @FXML
    private Label label; // injectnuto z FXML podle fx:id="label"

    @FXML
    private void handleButtonAction(ActionEvent event) {
        label.setText("Hello World!");
    }

    @Override
    public void initialize(URL url, ResourceBundle rb) {
        // Inicializace po načtení FXML
    }
}
```

4.6 Layouty v JavaFX

Layout	Použití
BorderPane	5 oblastí: top, bottom, left, right, center
HBox/VBox	Horizontální/vertikální řada
StackPane	Vrstvení komponent na sobě
GridPane	Mřížka s definicí řádků/sloupců
FlowPane	Zalamování jako FlowLayout
TilePane	Uniformní dlaždice
AnchorPane	Kotvení k okrajům

4.7 Životní cyklus JavaFX aplikace

1. `launch(args)` – statická metoda `Application`
2. Vytvoření instance aplikace
3. `init()` – inicializace (mimo UI vlákno!)
4. `start(Stage primaryStage)` – hlavní metoda, vytvoření scény
5. Běh aplikace – zpracování událostí
6. `stop()` – ukončení (volá se při zavření)

Vlákna v JavaFX

- `init()` – **spouštěcí vlákno** (ne UI!)
- `start()` a veškerá manipulace se scénou – **JavaFX Application Thread (JAT)**
- Náročné operace – **pracovní vlákna na pozadí** (`Task`, `Service`)

4.8 Pracovní vlákna v JavaFX

Třída	Účel
<code>Task<T></code>	Jednorázová úloha na pozadí, implementace <code>FutureTask</code>
<code>Service<T></code>	Opakované spouštění <code>Tasků</code> , sledování stavu
<code>WorkerStateEvent</code>	Události při změně stavu (SUCCEEDED, FAILED...)

```
Task<Integer> task = new Task<Integer>() {
    @Override
    protected Integer call() throws Exception {
        for (int i = 0; i < 100000; i++) {
            if (isCancelled()) break;
            updateProgress(i, 100000); // bezpečná aktualizace UI
        }
        return i;
    }
};
```

```
ProgressBar bar = new ProgressBar();
bar.progressProperty().bind(task.progressProperty());
new Thread(task).start();
```

5 Vlákna v Javě (Concurrency)

5.1 Procesy vs. vlákna

Aspekt	Proces	Vlákno
Paměť	Vlastní paměťový prostor	Sdílí paměť procesu
Zdroje	Samostatná množina	Sdílí zdroje procesu
Komunikace	Sokety, potrubí	Přímý přístup k paměti
Náročnost	Vysoká	Nízká (lightweight)

5.2 Vytvoření a spuštění vlákna

Dva způsoby:

1. **Runnable** – preferovaný, obecnější

```
public class HelloRunnable implements Runnable {
    public void run() { System.out.println("Hello from thread!"); }
    public static void main(String[] args) {
        new Thread(new HelloRunnable()).start();
    }
}
```

2. **Podtřída Thread** – přímo dědit, přepsat `run()`

```
public class HelloThread extends Thread {
    public void run() { ... }
    // new HelloThread().start();
}
```

Důležité!

Vlákno se spouští metodou `start()`, nikoli `run()`! `run()` by jen provedlo kód v aktuálním vlákně.

5.3 Stavy vlákna

Stav	Význam
NEW	Vytvořeno, ještě neběží
RUNNABLE	Připraveno ke běhu (může běžet, nemusí)
BLOCKED	Čeká na zámek (monitor)
WAITING	Čeká na notifikaci (<code>wait()</code> , <code>join()</code> bez timeoutu)
TIMED_WAITING	Čeká s časovým limitem (<code>sleep()</code> , <code>join(ms)</code>)
TERMINATED	Ukončeno (normálně nebo výjimkou)

Otázka ze vzorového testu!

Správná sekvence stavů: **NEW, Runnable, Blocked, Waiting, Timed waiting, Terminated**

Pozor na: Running není oficiální stav (je součástí Runnable), Stopped/Finished/Canceled nejsou standardní!

5.4 Přerušování vlákna (Interrupt)

Mechanismus interrupt

Přerušování je žádost, ne příkaz. Vláknko musí spolupracovat.

- `Thread.interrupt()` – nastaví příznak přerušování
- `Thread.interrupted()` – zkontroluje a **vymaže** příznak
- `isInterrupted()` – zkontroluje, **nevymaže**

```
// Správná podpora přerušování:
public void run() {
    while (!Thread.interrupted()) {
        // práce...
        try {
            Thread.sleep(1000); // vyhodí InterruptedException při interrupt
        } catch (InterruptedException e) {
            return; // nebo break, nebo obnovení příznaku
        }
    }
}
```

5.5 Synchronizace

5.5.1 Problémy souběhu

Thread Interference

Když více vláken přistupuje ke sdíleným datům a provádí operace, které nejsou **atomické**. I jednoduchá operace jako `c++` se přeloží na: načti `c`, zvýš, uloži.

Memory Consistency Errors

Různá vlákna mají různý pohled na sdílenou paměť. Řešením je relace **happens-before** – zaručená časová vazba mezi operacemi.

5.5.2 Nástroje synchronizace

Nástroj	Popis
<code>synchronized</code> metoda	Zámek na instanci (nebo třídě pro static). Jeden zámek na objekt – více synchronizovaných metod se vzájemně blokuje.
<code>synchronized</code> blok	Explicitní specifikace objektu zámku. Umožňuje jemnější granularitu (různé zámky pro různá data).
<code>ReentrantLock</code>	Explicitní zámek z <code>java.util.concurrent.locks</code> . Podporuje <code>tryLock()</code> (neblokující), podmínky (Condition).
Atomické proměnné	<code>AtomicInteger</code> , <code>AtomicLong</code> , <code>AtomicReference</code> – lock-free operace.

```
// Synchronizované metody -- jeden zámek na instanci
public synchronized void increment() { c++; }
```

```
public synchronized void decrement() { c--; }

// Synchronizované bloky -- různé zámky pro nezávislá data
private Object lock1 = new Object();
private Object lock2 = new Object();
public void inc1() { synchronized(lock1) { c1++; } }
public void inc2() { synchronized(lock2) { c2++; } }
```

Reentrant synchronizace

Vlákno může získat zámek, který **už vlastní** (např. synchronizovaná metoda volá jinou synchronizovanou metodu stejného objektu). Počítá se rekurzivně.

5.5.3 Guarded Blocks (wait/notify)

```
public synchronized void guardedJoy() {
    while(!joy) {
        try {
            wait(); // uvolní zámek, čeká na notify
        } catch (InterruptedException e) {}
    }
    // pokračování...
}

public synchronized notifyJoy() {
    joy = true;
    notifyAll(); // probudí VŠECHNA čekající vlákna
}
```

Pravidla pro wait/notify

- Vždy volej `wait()` uvnitř `while` cyklu, ne `if` (spurious wakeup!)
- Vždy používej `synchronized` blok/metodu při volání `wait/notify`
- Preferuj `notifyAll()` před `notify()` (spravedlnost)

5.6 Problémy souběhu – Deadlock, Starvation, Livelock

Problém	Popis
Deadlock	Dvě či více vláken navždy blokována, vzájemně čekají na zámky druhých. Příklad: vlákno A drží zámek 1, čeká na 2; vlákno B drží 2, čeká na 1.
Starvation	Vlákno nemůže získat přístup ke zdrojům, protože jiná vlákna je neustále získávají.
Livelock	Vlákna nejsou blokována, ale vzájemně reagují na akce druhých bez pokroku. Příklad: dva lidé se vyhýbají v chodbě, stále se posunují do stejné strany.

5.7 java.util.concurrent

5.7.1 Executor framework

ExecutorService

Odděluje **vytváření a management vláken** od aplikační logiky.

- **Executor** – základní rozhraní, metoda `execute(Runnable)`
- **ExecutorService** – rozšiřuje o `submit()`, `shutdown()`, `awaitTermination()`
- **ScheduledExecutorService** – plánované a opakované úlohy

```
// Thread pool s fixním počtem vláken
ExecutorService executor = Executors.newFixedThreadPool(4);

// Odeslání úlohy
Future<Long> result = executor.submit(new Callable<Long>() {
    public Long call() { return computeValue(); }
});

// Získání výsledku (blokující)
Long value = result.get();
executor.shutdown();
```

Otázka ze vzorového testu!

ExecutorService:

- Určuje, **jakým způsobem a v jakém vlákně** budou úlohy provedeny
- Poskytuje metody produkující **Future** pro asynchronní návratové hodnoty
- Zajišťuje provedení předložených úloh

Není pravda, že neurčuje způsob provedení – to je jeho hlavní účel!

5.7.2 Fork/Join framework

- Implementace **ExecutorService** pro **rekurzivní dělení** úloh
- **Work stealing** – volné vlákno “ukradne” práci z fronty jiného
- Základ: `ForkJoinPool`, `RecursiveTask<T>` (vrací hodnotu), `RecursiveAction`
- Metoda `compute()`: pokud je úloha malá, proved’ přímo; jinak rozděl a zavolej `invokeAll()`

5.7.3 Konkurentní kolekce

Třída/Interface	Účel
<code>BlockingQueue</code>	Fronta s blokujícím čtením/zápisem
<code>ArrayBlockingQueue</code>	Omezená kapacita
<code>ConcurrentHashMap</code>	Thread-safe <code>HashMap</code>
<code>ConcurrentSkipListMap</code>	Thread-safe <code>TreeMap</code>
<code>SynchronousQueue</code>	Předává prvky přímo (žádná kapacita)

5.8 Concurrency ve Swing

Tři druhy vláken ve Swing

1. **Initial threads** – počáteční vlákna (např. `main`)
2. **Event Dispatch Thread (EDT)** – zpracovává události, manipuluje s GUI komponentami
3. **Worker threads** – časově náročné úlohy na pozadí

Pravidlo EDT

Swingové komponenty nejsou thread-safe! Veškerá manipulace s GUI musí probíhat v **EDT**. Používej:

```
SwingUtilities.invokeLater(() -> { /* GUI kód */ });
SwingUtilities.invokeAndWait(() -> { /* GUI kód */ }); // blokující
```

5.8.1 SwingWorker

```
SwingWorker<ImageIcon[], Void> worker = new SwingWorker<>() {
    @Override
    protected ImageIcon[] doInBackground() throws Exception {
        // provádí se v pracovním vlákně
        return loadImages();
    }

    @Override
    protected void done() {
        // provádí se v EDT po skončení
        try {
            ImageIcon[] result = get();
            // aktualizace GUI...
        } catch (Exception e) { ... }
    }
};
worker.execute();
```

Generické parametry SwingWorker

`SwingWorker<T, V>`:

- `T` – návratový typ `doInBackground()` a `get()`
- `V` – typ mezivýsledků pro `publish()/process()`

6 Shrnutí klíčových otázek ze vzorového testu

6.1 Odpovědi na viditelné otázky

Otázka 1: Který typ výjimky pro neočekávané stavy? → Nekontrolované výjimky (`RuntimeException`)

Otázka 2: K čemu slouží `@FXML`? → Propojení identifikace komponent mezi kontrolerem a GUI

Otázka 3: Nejvyšší v hierarchii JavaFX? → `Stage`

- Otázka 4: Lze vyhodit výjimku v konstruktoru?** → Ano (bez omezení, není třeba uvádět v deklaraci pro unchecked; pro checked ano)
- Otázka 5: Co je defenzivní programování?** → Předvídání chyb a mimořádných stavů
- Otázka 6: Prostředky synchronizace:** → Synchronizované metody, synchronizované bloky (zámky jsou implementační detail, konstruktory se nesynchronizují)
- Otázka 7: Charakteristika ExecutorService:** → Určuje způsob a vlákno provedení, produkuje Future, zajišťuje provedení
- Otázka 8: Přístup vnitřní třídy k private členům?** → Ano – vnitřní třídy mají přístup ke všem členům vnější, i private
- Otázka 9: Co je anonymní vnitřní třída?** → Vnitřní třída, v jejíž deklaraci chybí hlavička (jméno)
- Otázka 10: Co provádí Event Dispatch Thread?** → Zpracovává systémovou frontu událostí týkajících se prvků GUI
- Otázka 11: Vnitřní třída a dědičnost:** → Vnitřní třída může být podtřídou vnější třídy (není to pravidlo, ale je to možné)
- Otázka 12: Stavy vlákna:** → New, Runnable, Blocked, Waiting, Timed waiting, Terminated
- Otázka 13: Výhody genericity:** → Čitelnější kód, kontroly při kompilaci (ne přetypování!)
- Otázka 14: Omezení typových parametrů:** → Omezuje množinu použitelných typových argumentů

7 Rychlá reference – často kladené otázky

Rozdíly Swing vs. JavaFX

Aspekt	Swing	JavaFX
Vlákno GUI	EDT	JavaFX Application Thread
Deklarativní GUI	Ne	FXML
CSS stylování	Omezené	Plná podpora
Scene Builder	Ne	Ano
3D grafika	Ne	Ano
WebView	Ne	Ano (WebKit)
Pracovní vlákna	SwingWorker	Task, Service

Kontrolní seznam před testem

- ☐ Rozdíl checked vs. unchecked exceptions
- ☐ Syntaxe try-catch-finally, vícenásobné catch
- ☐ throws klauzule, definice vlastních výjimek
- ☐ Účel assert (neplést s výjimkami!)
- ☐ Generické typy, wildcards, PECS pravidlo
- ☐ Type erasure – co se děje při překladu
- ☐ Hierarchie JavaFX: Stage > Scene > Pane > Node
- ☐ Účel @FXML anotace
- ☐ Vnořené třídy: statické vs. vnitřní, anonymní
- ☐ Stavy vlákn, životní cyklus
- ☐ Synchronizace: synchronized, ReentrantLock, atomické operace
- ☐ wait/notifyAll, Guarded Blocks
- ☐ Deadlock vs. Starvation vs. Livelock
- ☐ ExecutorService, ThreadPool, Future
- ☐ SwingWorker / Task + Service v JavaFX
- ☐ EDT vs. pracovní vlákna (proč oddělit GUI a výpočty)